

# **BÁO CÁO ĐỒ ÁN MÔN HỌC**

ĐỀ TÀI:

## **XÂY DỰNG HỆ THỐNG QUẢN LÝ DỰ ÁN PHẦN MỀM THEO MÔ HÌNH SCRUM**

*(Scrum Project Management Application)*

Sinh viên thực hiện: Trần Chí Thành

Mã số sinh viên: 20194845

Lớp / Khóa: ICT-02 K64

Giảng viên hướng dẫn: Nguyễn Đức Anh

Source code : <https://github.com/trnchthnh1301z/Graduation-Research-2>

# MỤC LỤC

|                                                              |    |
|--------------------------------------------------------------|----|
| MỤC LỤC.....                                                 | 2  |
| CHƯƠNG 1: TỔNG QUAN ĐỀ TÀI .....                             | 4  |
| 1.1. Bối cảnh và lý do chọn đề tài.....                      | 4  |
| 1.2. Mục tiêu đề tài.....                                    | 4  |
| 1.3. Phạm vi đồ án .....                                     | 4  |
| 1.4. Công nghệ sử dụng .....                                 | 5  |
| CHƯƠNG 2: PHÂN TÍCH YÊU CẦU .....                            | 6  |
| 2.1. Tác nhân hệ thống (Actors) .....                        | 6  |
| 2.2. Yêu cầu chức năng.....                                  | 6  |
| 2.2.1. Quản lý dự án (Project).....                          | 6  |
| 2.2.2. Quản lý Epic và Backlog .....                         | 6  |
| 2.2.3. Lập kế hoạch và thực thi Sprint .....                 | 6  |
| 2.2.4. Quản lý nguồn nhân lực (Resource).....                | 7  |
| 2.2.5. Quản lý chi phí (Cost).....                           | 7  |
| 2.2.6. Timeline tổng quan.....                               | 7  |
| 2.3. Yêu cầu phi chức năng.....                              | 7  |
| CHƯƠNG 3: THIẾT KẾ HỆ THỐNG.....                             | 8  |
| 3.1. Kiến trúc tổng thể .....                                | 8  |
| 3.2. Thiết kế cơ sở dữ liệu (Entity – Relationship) .....    | 8  |
| 3.3. Thiết kế API (REST endpoints).....                      | 10 |
| 3.4. Thiết kế giao diện (Frontend).....                      | 11 |
| CHƯƠNG 4: TRIỂN KHAI (IMPLEMENTATION).....                   | 12 |
| 4.1. Cấu trúc dự án Backend .....                            | 12 |
| 4.2. Một số logic nghiệp vụ tiêu biểu .....                  | 12 |
| 4.2.1. Ràng buộc loại trừ lẫn nhau (mutual exclusivity)..... | 12 |
| 4.2.2. Hoàn tất Sprint (Complete Sprint).....                | 12 |
| 4.2.3. Validate vị trí của Work Item (Location).....         | 12 |
| 4.3. Cấu trúc dự án Frontend.....                            | 12 |
| CHƯƠNG 5: KIỂM THỬ VÀ ĐÁNH GIÁ CHẤT LƯỢNG CODE .....         | 14 |
| 5.1. Unit test .....                                         | 14 |
| 5.2. Kết quả review code tĩnh .....                          | 14 |
| 5.3. Kiểm thử thực tế (chạy ứng dụng) .....                  | 14 |
| CHƯƠNG 6: KẾT QUẢ ĐẠT ĐƯỢC .....                             | 16 |
| 6.1. Trang Dashboard / Danh sách dự án .....                 | 16 |
| 6.2. Trang chi tiết dự án – Backlog.....                     | 16 |
| 6.3. Quản lý Sprint.....                                     | 18 |

|                                            |    |
|--------------------------------------------|----|
| 6.4. Quản lý nguồn lực và chi phí .....    | 18 |
| 6.5. Timeline tổng quan.....               | 19 |
| CHƯƠNG 7: HẠN CHẾ VÀ HƯỚNG PHÁT TRIỂN..... | 20 |
| 7.1. Hạn chế hiện tại .....                | 20 |
| 7.2. Hướng phát triển trong tương lai..... | 20 |
| KẾT LUẬN.....                              | 21 |
| TÀI LIỆU THAM KHẢO .....                   | 22 |

# CHƯƠNG 1: TỔNG QUAN ĐỀ TÀI

## 1.1. Bối cảnh và lý do chọn đề tài

Trong phát triển phần mềm hiện đại, mô hình Agile/Scrum được áp dụng rộng rãi nhờ khả năng thích ứng nhanh với thay đổi yêu cầu, tăng tính minh bạch và cải thiện hiệu quả cộng tác giữa các thành viên trong nhóm. Tuy nhiên, việc quản lý một dự án theo Scrum đòi hỏi phải theo dõi đồng thời nhiều yếu tố: tiến độ Sprint, hạng mục công việc (backlog), nguồn nhân lực, và chi phí phát sinh. Nếu không có công cụ hỗ trợ phù hợp, việc quản lý thủ công (qua bảng tính, tài liệu rời rạc) dễ dẫn đến sai lệch thông tin, khó tổng hợp báo cáo và thiếu khả năng theo dõi thời gian thực.

Xuất phát từ thực tế đó, đề án này xây dựng một ứng dụng web quản lý dự án phần mềm theo mô hình Scrum, cho phép người dùng quản lý vòng đời dự án từ giai đoạn lập kế hoạch (Epic, Sprint) đến thực thi (Backlog, Work Item) và giám sát nguồn lực, chi phí một cách tập trung.

## 1.2. Mục tiêu đề tài

Đề án hướng đến các mục tiêu cụ thể sau:

- Xây dựng hệ thống quản lý dự án (project) cho phép tạo, theo dõi nhiều dự án song song, mỗi dự án có trạng thái riêng (Planning, Active, Completed...).
- Hỗ trợ phân rã công việc theo cấu trúc Epic → Work Item (Story/Task/Bug), tổ chức backlog theo Sprint.
- Cho phép lập kế hoạch Sprint (Sprint Planning) và theo dõi tiến độ thực thi theo trạng thái (To Do / In Progress / Done).
- Quản lý nguồn nhân lực: gán thành viên (Person) vào Epic/Work Item, theo dõi workload theo từng Sprint.
- Quản lý chi phí: định nghĩa khoản chi phí (Cost) và gán cho Epic/Work Item, tổng hợp chi phí theo dự án/sprint.
- Cung cấp giao diện trực quan (dashboard, timeline) giúp người quản lý dự án (PM/PO) có cái nhìn tổng thể về tiến độ và nguồn lực.
- Thiết kế kiến trúc backend RESTful rõ ràng, có khả năng mở rộng, tách biệt các lớp (model – DTO – mapper – service – controller).

## 1.3. Phạm vi đề án

Phạm vi của đề án tập trung vào việc xây dựng một ứng dụng full-stack hoàn chỉnh gồm hai phần:

- Backend: cung cấp REST API phục vụ các nghiệp vụ CRUD và nghiệp vụ chuyên biệt (hoàn tất Sprint, validate ràng buộc dữ liệu...).
- Frontend: giao diện web cho phép người dùng tương tác trực tiếp với các chức năng trên thông qua trình duyệt.

Đồ án giả định chạy trong môi trường phát triển cục bộ (local development), sử dụng cơ sở dữ liệu nhúng H2 cho mục đích demo, có khả năng chuyển sang MySQL cho môi trường triển khai thực tế. Đồ án không bao gồm các chức năng: xác thực/phân quyền chi tiết theo vai trò ở mức bảo mật production, thông báo real-time, hoặc tích hợp CI/CD.

## 1.4. Công nghệ sử dụng

| Thành phần                | Công nghệ                                                  |
|---------------------------|------------------------------------------------------------|
| Ngôn ngữ Backend          | Java 21                                                    |
| Framework Backend         | Spring Boot 3 (Spring Web/MVC, Spring Data JPA, Hibernate) |
| Build tool Backend        | Maven (Maven Wrapper)                                      |
| Cơ sở dữ liệu             | H2 (môi trường dev), MySQL (môi trường production)         |
| Thư viện hỗ trợ           | Lombok (giảm boilerplate code: getter/setter/builder)      |
| Ngôn ngữ Frontend         | TypeScript                                                 |
| Framework Frontend        | React 18                                                   |
| Build tool Frontend       | Vite                                                       |
| UI Framework              | TailwindCSS kết hợp shadcn/ui (dựa trên Radix UI)          |
| Quản lý điều hướng        | React Router                                               |
| Giao tiếp API             | Axios                                                      |
| Quản lý state/server-data | TanStack Query (React Query)                               |

## CHƯƠNG 2: PHÂN TÍCH YÊU CẦU

### 2.1. Tác nhân hệ thống (Actors)

Hệ thống được thiết kế hướng tới ba nhóm vai trò người dùng chính, mỗi vai trò tương ứng với một tập quyền hạn và nhu cầu sử dụng khác nhau:

| Vai trò                         | Mô tả quyền hạn / nhu cầu                                                                                                                                |
|---------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Project Manager / Product Owner | Tạo và quản lý dự án, định nghĩa Epic, lập kế hoạch Sprint, theo dõi tiến độ và chi phí tổng thể của dự án.                                              |
| Team Member                     | Cập nhật trạng thái Work Item được giao (To Do → In Progress → Done), xem backlog và lịch Sprint của mình.                                               |
| Admin / PMO                     | Quản lý danh mục dùng chung trong toàn hệ thống: danh sách People (nhân sự) và danh mục Cost item (loại chi phí), không gắn riêng theo một dự án cụ thể. |

### 2.2. Yêu cầu chức năng

#### 2.2.1. Quản lý dự án (Project)

- Tạo mới, xem danh sách, xem chi tiết, cập nhật, xóa một dự án.
- Mỗi dự án có thuộc tính: tiêu đề, mô tả, trạng thái (PLANNING / ACTIVE / COMPLETED...).
- Trang Dashboard hiển thị danh sách dự án kèm số liệu tổng hợp: số Epic, số thành viên tham gia, tổng chi phí ước tính.
- Cho phép lọc danh sách dự án theo trạng thái.

#### 2.2.2. Quản lý Epic và Backlog

- Tạo, sửa, xóa Epic trong một dự án; mỗi Epic có khoảng thời gian thực hiện (startDate – endDate).
- Tạo, sửa, xóa Work Item (Story/Task/Bug) gắn với một Epic; phân loại theo loại (type), độ ưu tiên (priority) và trạng thái (status).
- Hiển thị bảng Backlog cho phép lọc Work Item theo Sprint, theo Epic hoặc theo trạng thái.

#### 2.2.3. Lập kế hoạch và thực thi Sprint

- Tạo Sprint mới gắn với một dự án, có thời gian bắt đầu/kết thúc xác định (mặc định chu kỳ 2 tuần theo Scrum thực tế).
- Gán Work Item vào Sprint (chuyển từ vị trí Backlog sang Sprint).
- Theo dõi tiến độ Sprint qua trạng thái từng Work Item (TO\_DO / IN\_PROGRESS / DONE).
- Hoàn tất Sprint (complete sprint): các Work Item chưa hoàn thành được chuyển sang một Sprint khác hoặc đưa về Backlog.

#### 2.2.4. Quản lý nguồn nhân lực (Resource)

- Quản lý danh mục People dùng chung toàn hệ thống.
- Gán (assign) một hoặc nhiều People vào Epic hoặc Work Item cụ thể.
- Xem workload (khối lượng công việc) của từng người theo Sprint.

#### 2.2.5. Quản lý chi phí (Cost)

- Quản lý danh mục các khoản mục chi phí (Cost item) dùng chung toàn hệ thống.
- Gán chi phí cho Epic hoặc Work Item (một khoản chi phí chỉ gán với một trong hai – không gán đồng thời cả hai).
- Tổng hợp chi phí theo dự án và theo Sprint.

#### 2.2.6. Timeline tổng quan

- Hiện thị lịch trình tổng thể các Sprint trong một dự án, giúp người quản lý nắm bắt mốc thời gian quan trọng.

### 2.3. Yêu cầu phi chức năng

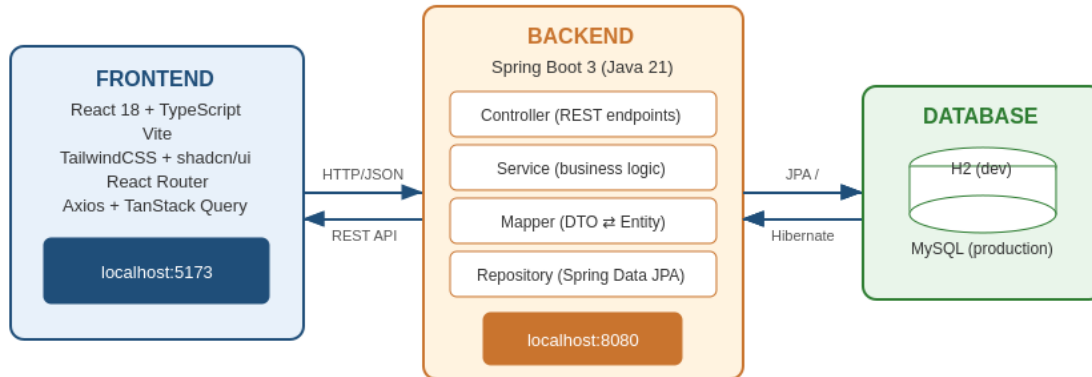
- Kiến trúc backend phải phân tách rõ các lớp (layered architecture) để dễ kiểm thử và mở rộng.
- API tuân theo chuẩn RESTful, trả về dữ liệu dạng JSON, sử dụng đúng HTTP method (GET/POST/PUT/DELETE) theo từng nghiệp vụ.
- Giao diện responsive, sử dụng hệ thống design token nhất quán (TailwindCSS + shadcn/ui).
- Dữ liệu nghiệp vụ phải được validate chặt chẽ ở tầng entity và tầng service (ví dụ: một Cost/Person assignment chỉ được gán với Epic HOẶC Work Item, không gán cả hai).
- Hệ thống có khả năng chạy độc lập trong môi trường phát triển cục bộ, dùng H2 in-memory/file-based, không phụ thuộc dịch vụ ngoài.

## CHƯƠNG 3: THIẾT KẾ HỆ THỐNG

### 3.1. Kiến trúc tổng thể

Hệ thống được xây dựng theo kiến trúc client – server, tách biệt hoàn toàn giữa Frontend (Single Page Application) và Backend (REST API server), giao tiếp với nhau thông qua HTTP/JSON.

Sơ đồ kiến trúc tổng thể hệ thống



Mô hình Client – Server: Frontend (SPA) giao tiếp với Backend qua REST API, Backend truy xuất dữ liệu qua JPA/Hibernate

Backend được tổ chức theo mô hình phân tầng (layered architecture) chuẩn của Spring Boot, gồm các lớp:

| Tầng (package) | Vai trò                                                                                                                              |
|----------------|--------------------------------------------------------------------------------------------------------------------------------------|
| model          | Định nghĩa các entity JPA, ánh xạ trực tiếp tới bảng trong cơ sở dữ liệu.                                                            |
| dto            | Data Transfer Object – đối tượng dữ liệu trao đổi giữa client và server, tách biệt với entity nội bộ.                                |
| mapper         | Chuyển đổi qua lại giữa Entity và DTO, đồng thời resolve các khóa ngoại (project/sprint/epic id) thông qua repository.               |
| repository     | Interface Spring Data JPA, cung cấp các phương thức truy vấn dữ liệu.                                                                |
| service        | Chứa toàn bộ business logic: validate nghiệp vụ, điều phối giữa các repository, xử lý các use case phức tạp (ví dụ hoàn tất Sprint). |
| controller     | Định nghĩa REST endpoint, nhận request, gọi service và trả response.                                                                 |
| constants      | Định nghĩa các enum dùng chung: trạng thái (status), độ ưu tiên (priority), loại Work Item (type).                                   |
| config         | Cấu hình ứng dụng, bao gồm DataSeeder để khởi tạo dữ liệu mẫu khi chạy ở profile dev.                                                |

### 3.2. Thiết kế cơ sở dữ liệu (Entity – Relationship)

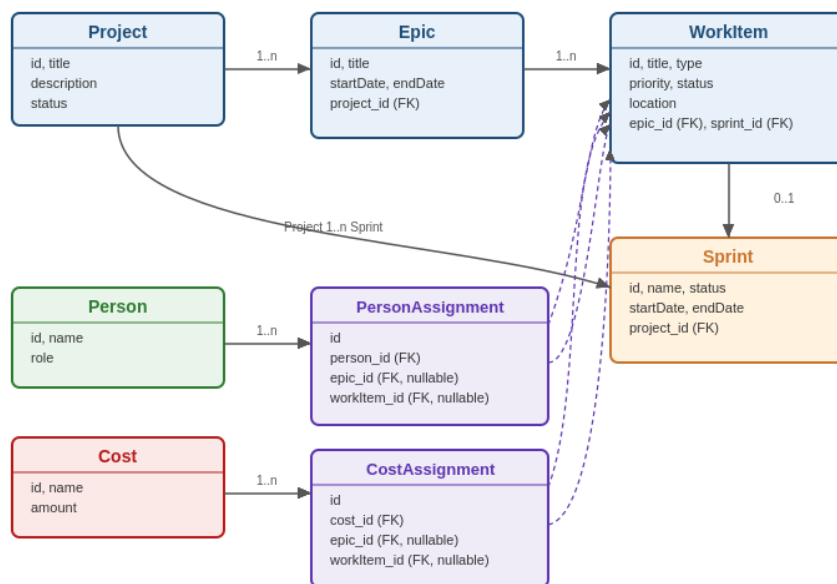
Mô hình dữ liệu xoay quanh thực thể trung tâm Project, được phân rã theo cấu trúc Scrum: Project chứa nhiều Epic, Epic chứa nhiều Work Item, Work Item có thể được gán vào một



Sprint. Nguồn lực (Person) và chi phí (Cost) được quản lý dùng chung và gán linh hoạt vào Epic hoặc Work Item thông qua các bảng quan hệ trung gian.

| Entity           | Mô tả                                                                                                              |
|------------------|--------------------------------------------------------------------------------------------------------------------|
| Project          | Dự án – thực thể gốc, chứa danh sách Epic và Sprint.                                                               |
| Epic             | Tính năng/mục tiêu lớn của dự án, có khoảng thời gian thực hiện, chứa nhiều Work Item.                             |
| WorkItem         | Đơn vị công việc cụ thể (Story/Task/Bug) thuộc một Epic, có thể được gán vào một Sprint hoặc giữ ở vị trí Backlog. |
| Sprint           | Chu kỳ phát triển có thời hạn xác định, chứa danh sách Work Item được lập kế hoạch thực hiện trong chu kỳ đó.      |
| Person           | Thành viên nhóm, dùng chung toàn hệ thống, được gán vào Epic/Work Item qua PersonAssignment.                       |
| Cost             | Khoản mục chi phí, dùng chung toàn hệ thống, được gán vào Epic/Work Item qua CostAssignment.                       |
| PersonAssignment | Bảng quan hệ trung gian: gán một Person vào một Epic HOẶC một Work Item (ràng buộc loại trừ lẫn nhau).             |
| CostAssignment   | Bảng quan hệ trung gian: gán một Cost vào một Epic HOẶC một Work Item (ràng buộc loại trừ lẫn nhau).               |

Sơ đồ ERD - Mô hình dữ liệu hệ thống



Lưu ý: PersonAssignment và CostAssignment chỉ gán với Epic HOẶC WorkItem (loại trừ lẫn nhau – mutual exclusivity), được validate ở cả tầng entity (@PrePersist/@PreUpdate) và tầng service trước khi lưu xuống cơ sở dữ liệu.

Một điểm thiết kế đáng chú ý: PersonAssignment và CostAssignment đều áp dụng ràng buộc "loại trừ lẫn nhau" (mutual exclusivity) — một assignment chỉ được gán với Epic hoặc Work Item, không được gán đồng thời cả hai, và ràng buộc này được kiểm tra ở hai lớp: tại tầng entity (thông qua callback @PrePersist/@PreUpdate của JPA) và tại tầng service (trước khi gọi

repository lưu dữ liệu), đảm bảo tính nhất quán dữ liệu ngay cả khi có thao tác ghi trực tiếp ngoài luồng service thông thường.

### 3.3. Thiết kế API (REST endpoints)

Các controller cung cấp REST API theo từng nhóm chức năng tương ứng với entity, tuân theo các quy ước RESTful chuẩn:

| Resource                             | Method / Path mẫu                      | Chức năng                                                   |
|--------------------------------------|----------------------------------------|-------------------------------------------------------------|
| Project                              | GET/POST/PUT/DELETE<br>/api/projects   | CRUD dự án                                                  |
| Epic                                 | GET/POST/PUT/DELETE<br>/api/epics      | CRUD Epic, xóa cascade các dữ liệu liên quan                |
| WorkItem                             | GET/POST/PUT/DELETE<br>/api/work-items | CRUD Work Item, cập nhật trạng thái/vị trí (sprint/backlog) |
| Sprint                               | GET/POST/PUT/DELETE<br>/api/sprints    | CRUD Sprint, endpoint riêng để hoàn tất Sprint (complete)   |
| Person                               | GET/POST/PUT/DELETE<br>/api/people     | CRUD nhân sự dùng chung                                     |
| Cost                                 | GET/POST/PUT/DELETE<br>/api/costs      | CRUD khoản mục chi phí dùng chung                           |
| PersonAssignment /<br>CostAssignment | POST/DELETE<br>/api/.../assignments    | Gán/hủy gán nguồn lực và chi phí vào Epic/Work Item         |

### 3.4. Thiết kế giao diện (Frontend)

Frontend được tổ chức theo cấu trúc feature-based (mỗi nghiệp vụ là một thư mục feature riêng), giúp tách biệt rõ ràng logic của từng module:

| Module (features/)       | Chức năng giao diện                                                                                                              |
|--------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| Landing / Dashboard      | Trang chủ hiển thị danh sách dự án dạng card, kèm số liệu tổng hợp (số Epic, số thành viên, chi phí), có bộ lọc theo trạng thái. |
| project (chi tiết dự án) | Trang chi tiết một dự án, là khung chứa các tab con: Backlog, Sprint, Resource, Cost, Timeline.                                  |
| epic / backlog           | Quản lý Epic và bảng Work Item dạng backlog, có thể lọc theo Sprint/trạng thái.                                                  |
| sprint                   | Tạo/sửa Sprint, xem tiến độ thực thi của các Work Item trong Sprint.                                                             |
| resource                 | Danh sách People và workload theo Sprint.                                                                                        |
| cost                     | Quản lý khoản mục chi phí và tổng hợp chi phí theo Epic/Sprint/Project.                                                          |
| timeline                 | Hiển thị tổng quan lịch các Sprint trong dự án.                                                                                  |

Về mặt UI component, dự án sử dụng shadcn/ui (xây trên Radix UI Primitives) kết hợp TailwindCSS để đảm bảo tính nhất quán về thiết kế (button, badge trạng thái, card, dialog, table...) mà vẫn giữ khả năng tùy biến cao.

## CHƯƠNG 4: TRIỂN KHAI (IMPLEMENTATION)

### 4.1. Cấu trúc dự án Backend

```

project-management-be/
├── src/main/java/com/thesis/projectmanagement/
│   ├── model/          (Project, Epic, WorkItem, Sprint, Person, Cost, ...)
│   ├── dto/
│   ├── mapper/
│   ├── repository/
│   ├── service/        (ProjectService, EpicService, SprintService, ...)
│   ├── controller/     (ProjectController, SprintController, ...)
│   ├── constants/      (các enum: status, priority, type)
│   ├── config/         (DataSeeder)
│   └── ProjectManagementApplication.java
├── src/test/java/... (unit test theo từng model)
├── pom.xml
└── mvnw / mvnw.cmd
  
```

### 4.2. Một số logic nghiệp vụ tiêu biểu

#### 4.2.1. Ràng buộc loại trừ lẫn nhau (mutual exclusivity)

Đối với CostAssignment và PersonAssignment, hệ thống đảm bảo một assignment chỉ gắn với Epic hoặc Work Item, không gắn cả hai cùng lúc. Logic này được kiểm tra hai lớp độc lập (entity-level callback và service-level validate trước khi persist), tránh được trường hợp dữ liệu không hợp lệ lọt qua nếu một lớp bị bỏ sót khi mở rộng tính năng sau này.

#### 4.2.2. Hoàn tất Sprint (Complete Sprint)

Khi một Sprint được đánh dấu hoàn tất, hệ thống thu thập danh sách các Work Item chưa đạt trạng thái DONE (incomplete items), sau đó cho phép chuyển các Work Item này sang một Sprint khác hoặc đưa về vị trí Backlog. Đây là nghiệp vụ phản ánh đúng thực tế vận hành Scrum: các hạng mục chưa hoàn thành trong một Sprint thường được "carry-over" sang Sprint tiếp theo.

#### 4.2.3. Validate vị trí của Work Item (Location)

Một Work Item có thể tồn tại ở một trong các vị trí: BACKLOG, SPRINT hoặc COMPLETED. Hệ thống validate tính nhất quán giữa trường vị trí (location) và việc Work Item có đang được gán vào Sprint hay không, đảm bảo không xảy ra trạng thái mâu thuẫn (ví dụ: vị trí là BACKLOG nhưng vẫn còn liên kết tới một Sprint cụ thể).

### 4.3. Cấu trúc dự án Frontend

```

project-management-fe/
├── src/
│   ├── features/
│   │   ├── project/ (backlog, sprint, resource, cost, timeline)
│   │   └── epic/
│   ├── lib/
│   │   ├── api.ts    (Axios instance + interceptor xử lý lỗi)
│   │   └── types.ts  (định nghĩa type/interface tương ứng DTO backend)
│   ├── components/ui/ (shadcn/ui components)
│   └── App.tsx / main.tsx
  
```

 package.json  
vite.config.ts

Tầng giao tiếp API (lib/api.ts) sử dụng Axios với interceptor tập trung để xử lý lỗi response, kết hợp với TanStack Query để quản lý cache và trạng thái loading/error khi gọi dữ liệu từ backend, giúp giảm thiểu code lặp lại ở từng component.

## CHƯƠNG 5: KIỂM THỬ VÀ ĐÁNH GIÁ CHẤT LƯỢNG CODE

### 5.1. Unit test

Dự án đã viết unit test cho các model chính (ProjectTest, EpicTest, SprintTest, WorkItemTest, PersonTest...), tập trung kiểm tra: builder pattern, các quan hệ hai chiều giữa entity (ví dụ thêm Work Item vào Epic phải đồng thời thiết lập ngược lại Work Item.epic), và logic equals/hashCode.

### 5.2. Kết quả review code tĩnh

Trong quá trình review code tĩnh (không chạy thực tế), đã phát hiện một số vấn đề đáng chú ý như sau:

| Mức độ        | Vấn đề                                                                                                                                                                                                                                     |
|---------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Lỗi biên dịch | EpicTest.java: builder CostAssignment.builder().epic() bị gọi thiếu tham số (.epic() thay vì .epic(epic)), khiến project không thể build (test-compile failure). Đã được sửa trong quá trình thực hiện đồ án.                              |
| Bug logic     | SprintService.createSprint(): trạng thái Sprint (status) được set SAU khi gọi repository.save(), nên giá trị lưu thực tế trong CSDL có thể không đúng như kỳ vọng (DTO trả về cho client có vẻ đúng nhưng dữ liệu lưu trữ chưa chắc khớp). |
| Thiết kế      | Toàn bộ backend chưa có lớp xử lý lỗi tập trung (@ControllerAdvice), khiến các lỗi nghiệp vụ (not-found, vi phạm ràng buộc) đều trả về HTTP 500 thay vì mã lỗi phù hợp (404/400) theo chuẩn REST.                                          |
| Cấu hình      | DataSeeder được đánh dấu @Profile("dev") nhưng không có profile nào được set là active theo mặc định trong application.properties, nên dữ liệu mẫu sẽ không được tự sinh nếu chạy ứng dụng ở chế độ mặc định.                              |
| Nhỏ           | Một số thông báo lỗi (exception message) trong WorkItem.validateLocation() chưa diễn đạt đúng logic thực tế, có thể gây nhầm lẫn khi debug.                                                                                                |

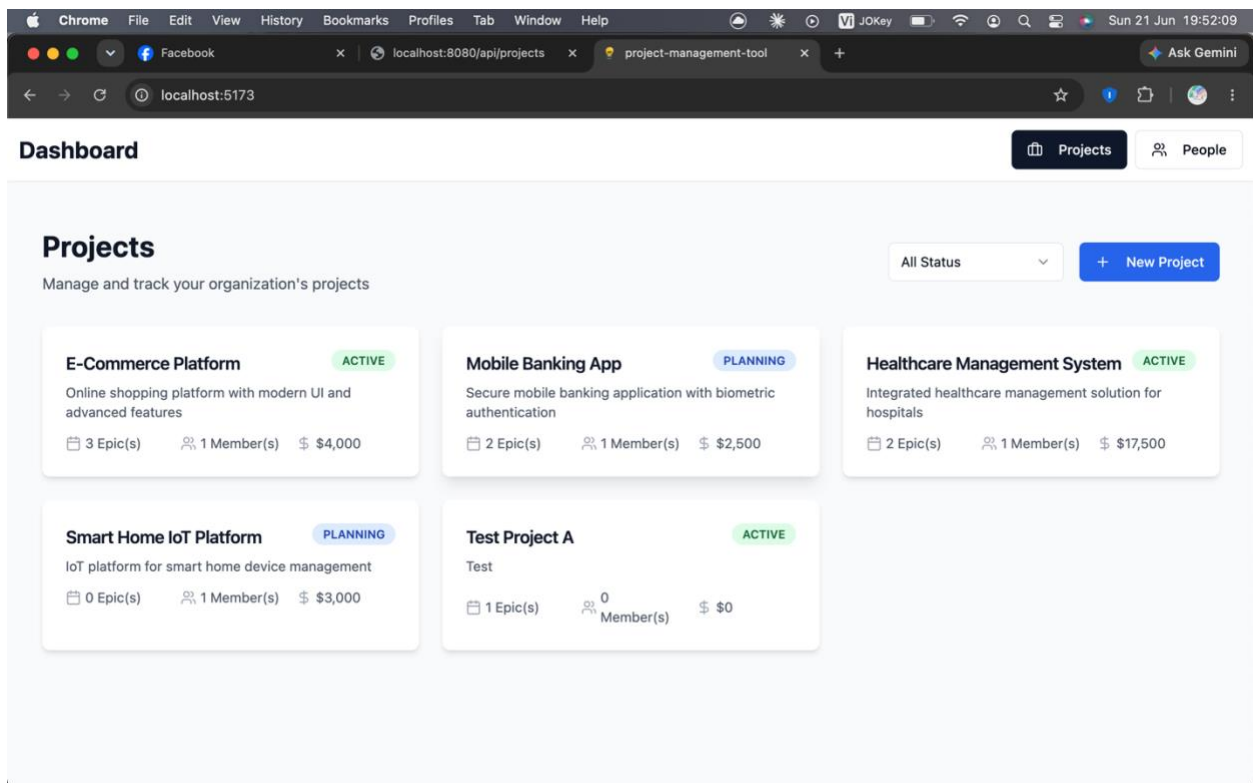
### 5.3. Kiểm thử thực tế (chạy ứng dụng)

Sau khi cấu hình đúng môi trường (JDK 21, Node.js 20), ứng dụng đã được build và chạy thành công trên môi trường local:

- Backend (Spring Boot) chạy ổn định tại http://localhost:8080, API trả đúng dữ liệu JSON cho các endpoint dự án.
- Frontend (Vite + React) chạy tại http://localhost:5173, giao diện kết nối thành công với backend, hiển thị đúng danh sách dự án kèm số liệu tổng hợp.

```

[{"id":31,"title":"E-Commerce Platform","description":"Online shopping platform with modern UI and advanced features","status":"ACTIVE"}, {"id":32,"title":"Mobile Banking App","description":"Secure mobile banking application with biometric authentication","status":"PLANNING"}, {"id":33,"title":"Healthcare Management System","description":"Integrated healthcare management solution for hospitals","status":"ACTIVE"}, {"id":34,"title":"Smart Home IoT Platform","description":"IoT platform for smart home device management","status":"PLANNING"}, {"id":35,"title":"Test Project A","description":"Test","status":"ACTIVE"}]
  
```

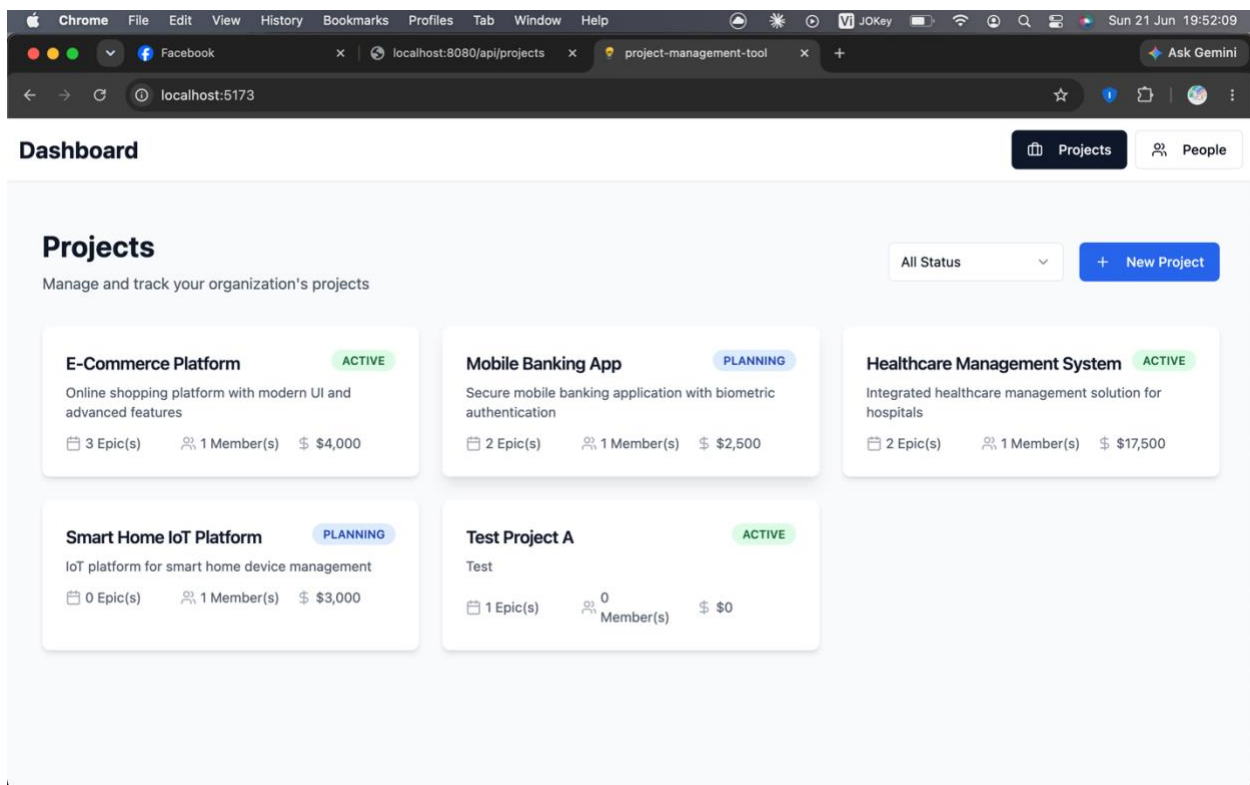


## CHƯƠNG 6: KẾT QUẢ ĐẠT ĐƯỢC

Sau khi hoàn thành, hệ thống đã đạt được các kết quả cụ thể sau, được minh họa qua các màn hình thực tế:

### 6.1. Trang Dashboard / Danh sách dự án

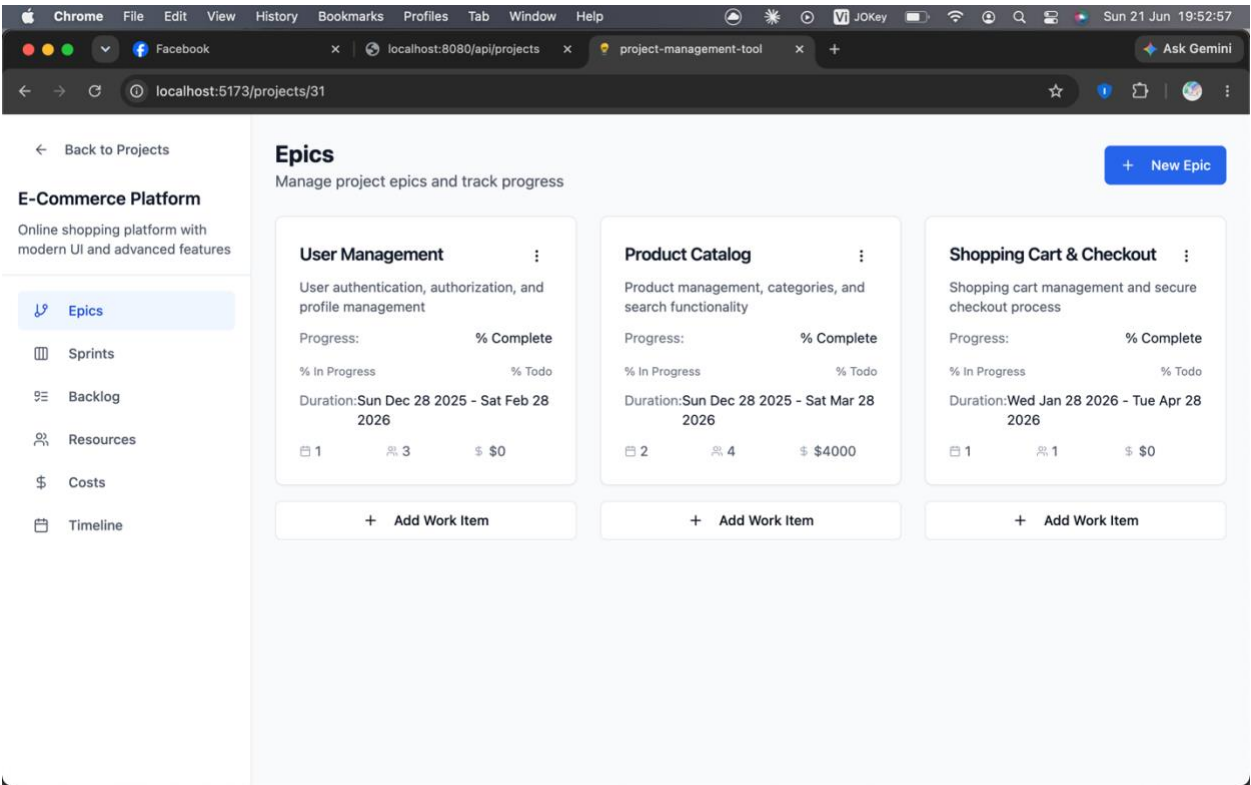
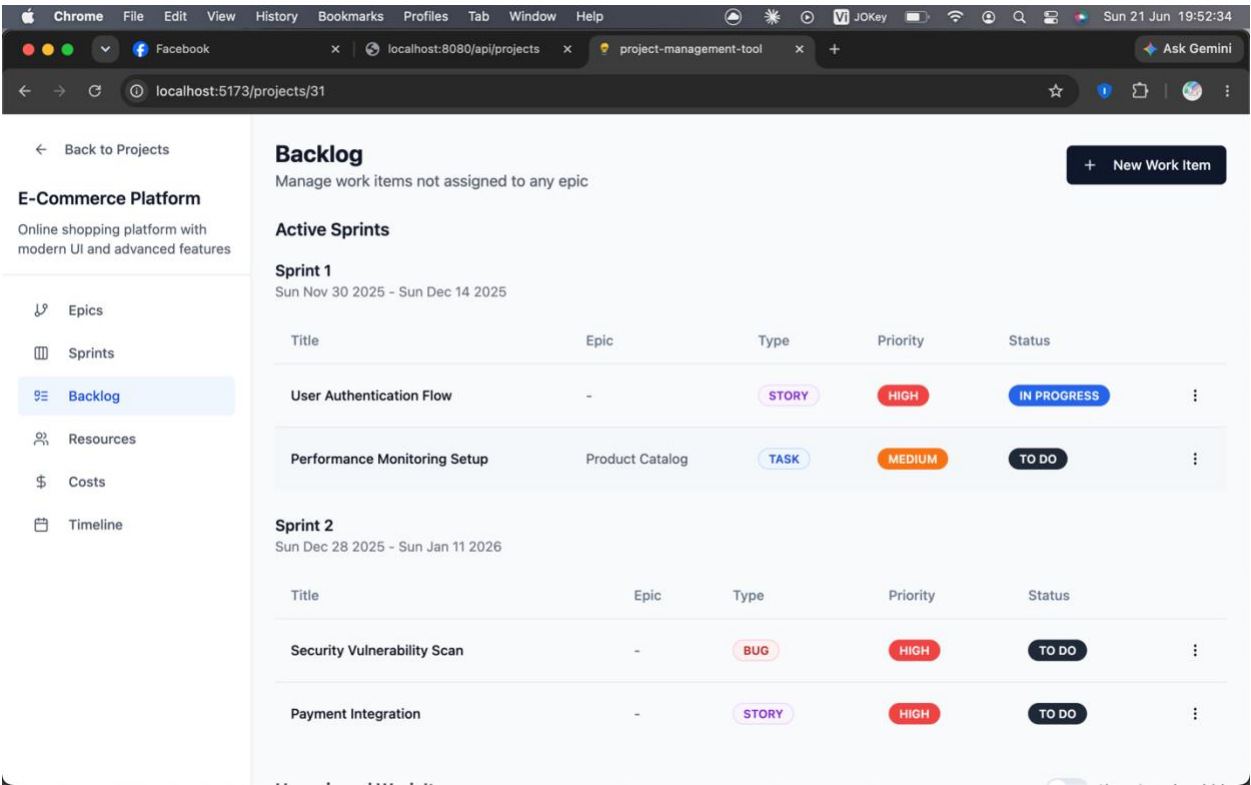
Trang chủ hiển thị toàn bộ danh sách dự án dưới dạng card, mỗi card thể hiện: tên dự án, trạng thái (badge màu sắc phân biệt ACTIVE/PLANNING/...), mô tả ngắn, số Epic, số thành viên tham gia và tổng chi phí ước tính. Người dùng có thể lọc theo trạng thái và tạo dự án mới qua nút "New Project".



### 6.2. Trang chi tiết dự án – Backlog

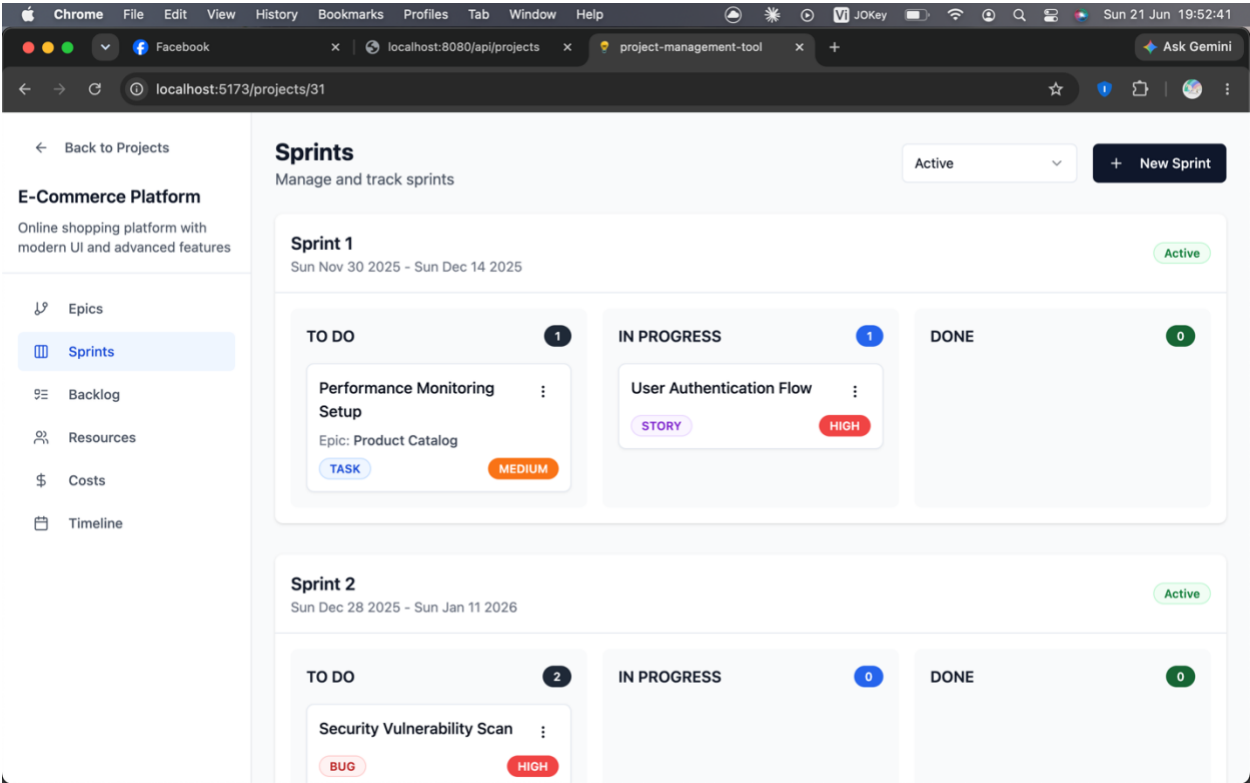
Trang chi tiết một dự án cụ thể, hiển thị danh sách Epic và bảng Work Item dạng backlog.



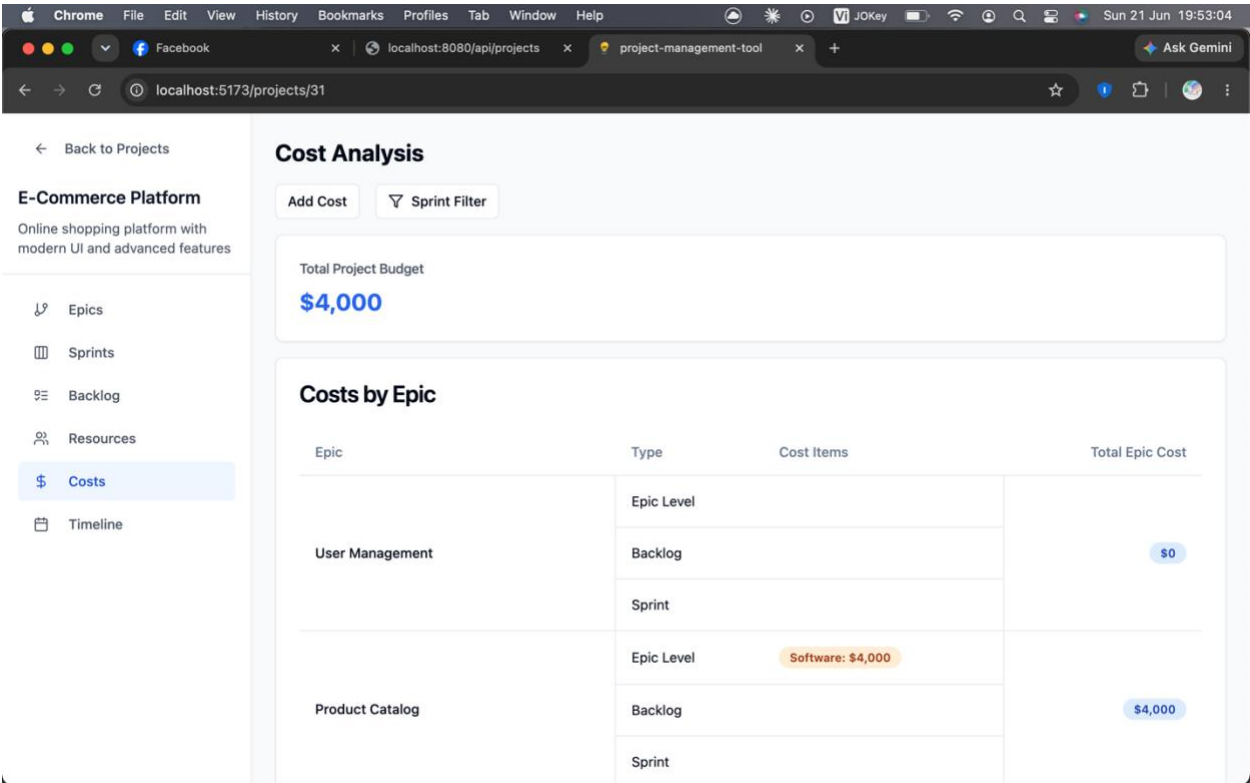


6.3. Quản lý Sprint

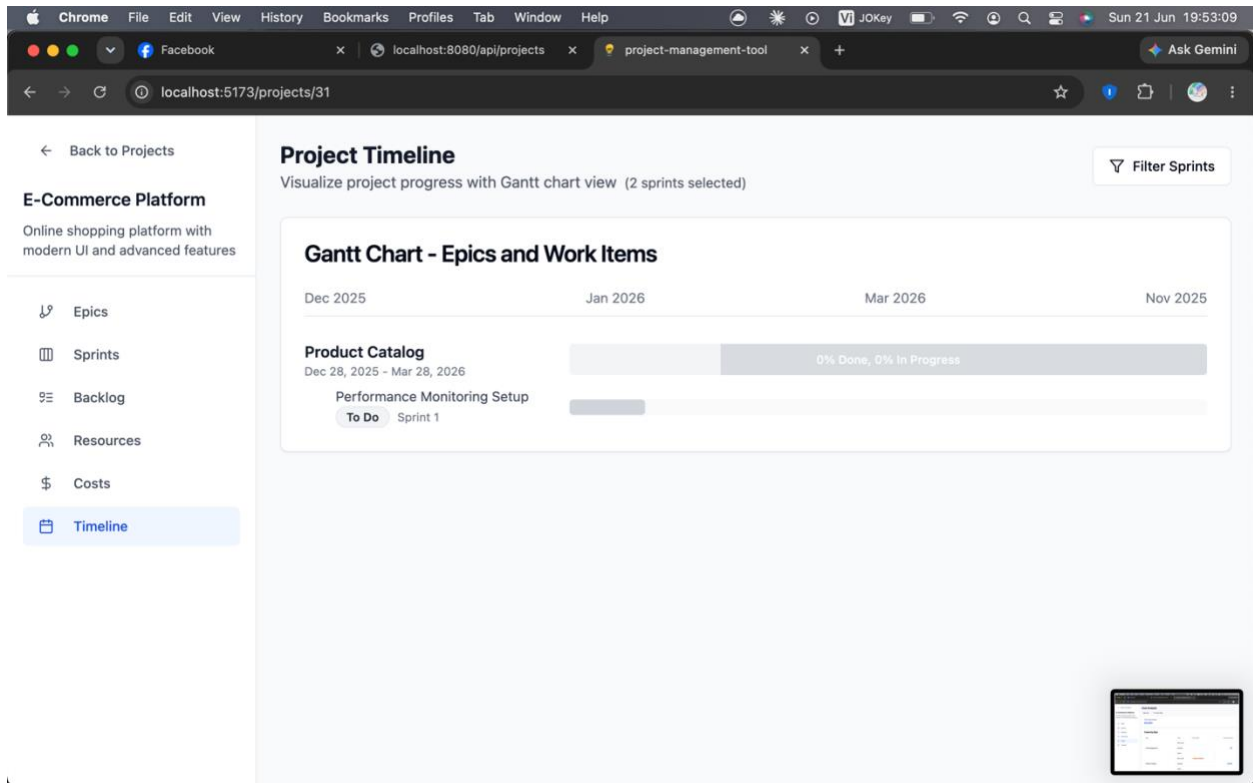
Giao diện cho phép tạo Sprint mới, xem danh sách Work Item theo từng trạng thái thực thi.



6.4. Quản lý nguồn lực và chi phí



## 6.5. Timeline tổng quan



## CHƯƠNG 7: HẠN CHẾ VÀ HƯỚNG PHÁT TRIỂN

### 7.1. Hạn chế hiện tại

- Chưa có lớp xử lý lỗi tập trung (Global Exception Handler), dẫn đến mã lỗi HTTP trả về chưa đúng chuẩn REST trong một số trường hợp.
- Chưa có cơ chế xác thực/phân quyền (authentication/authorization) thực sự – các vai trò người dùng (PM/PO, Team Member, Admin) hiện chỉ mang tính khái niệm thiết kế, chưa được hiện thực hóa bằng cơ chế đăng nhập, JWT hay session.
- Bug ở `SprintService.createSprint()` khiến trạng thái Sprint khi tạo mới có thể không được lưu đúng vào cơ sở dữ liệu (đã ghi nhận ở Chương 5, cần sửa trước khi đưa vào production).
- DataSeeder chưa active theo mặc định, gây nhầm lẫn khi người dùng mới chạy ứng dụng lần đầu mà không biết cần bật profile dev.
- Chưa có kiểm thử tích hợp (integration test) ở tầng controller/service, mới chỉ có unit test ở tầng model.

### 7.2. Hướng phát triển trong tương lai

- Bổ sung `@ControllerAdvice` để xử lý lỗi tập trung, trả về mã HTTP và message chuẩn hóa (404 Not Found, 400 Bad Request, 409 Conflict...).
- Tích hợp xác thực người dùng (Spring Security + JWT) để hiện thực hóa đầy đủ phân quyền theo vai trò đã thiết kế.
- Bổ sung thông báo real-time (WebSocket) khi có thay đổi trạng thái Work Item/Sprint, hỗ trợ cộng tác nhóm tốt hơn.
- Viết thêm integration test và thiết lập CI/CD pipeline (GitHub Actions) để tự động kiểm thử và build khi có thay đổi code.
- Triển khai ứng dụng lên môi trường cloud (ví dụ Render, Railway cho backend; Vercel/Netlify cho frontend) để demo trực tuyến.
- Bổ sung biểu đồ trực quan (Burndown chart, Velocity chart) hỗ trợ phân tích hiệu suất Sprint theo đúng thông lệ Scrum.

## KẾT LUẬN

Đồ án đã xây dựng thành công một ứng dụng web full-stack quản lý dự án phần mềm theo mô hình Scrum, với backend Spring Boot cung cấp REST API có kiến trúc phân tầng rõ ràng, và frontend React/TypeScript cung cấp giao diện trực quan, hiện đại. Hệ thống đáp ứng được các nghiệp vụ cốt lõi của Scrum: quản lý Epic/Backlog, lập kế hoạch và thực thi Sprint, quản lý nguồn lực và chi phí.

Thông qua quá trình review code tĩnh và kiểm thử thực tế, đồ án cũng đã chỉ ra được một số hạn chế cụ thể (bug logic, thiếu xử lý lỗi tập trung, thiếu cơ chế xác thực) làm cơ sở cho các hướng cải tiến trong tương lai. Quá trình triển khai và chạy thử ứng dụng trên môi trường thực tế (cài đặt JDK 21, Node.js 20, debug các vấn đề về môi trường) cũng giúp hiểu rõ hơn về quy trình DevOps cơ bản khi đưa một ứng dụng full-stack từ source code đến trạng thái chạy được.

## TÀI LIỆU THAM KHẢO

1. Spring Boot Reference Documentation – <https://docs.spring.io/spring-boot/>
2. Spring Data JPA Reference Documentation – <https://docs.spring.io/spring-data/jpa/>
3. React Documentation – <https://react.dev/>
4. Vite Documentation – <https://vitejs.dev/>
5. shadcn/ui Documentation – <https://ui.shadcn.com/>
6. TanStack Query Documentation – <https://tanstack.com/query/>
7. Scrum Guide – Ken Schwaber & Jeff Sutherland, <https://scrumguides.org/>